LEAKAGE-AUDITED MATRIX-FREE RESIDUAL PROPAGATION FOR GENE MEASUREMENTS

Molena Huynh
North Carolina State University
molena.huynh@jmp.com

Abstract. We revisit held-out marker-gene prediction on public single-cell count matrices after treating the previous algorithm, comparisons, tables, and figures as untrusted. The audit changes the conclusion. A tuned ExtraTrees model reaches lower error than the former graph method, and conventional total-count normalization leaks the held-out target counts through the library-size denominator. We therefore replace the method by cross-fitted matrix-free residual propagation (CF-MFRP): normalize by non-target molecules, fit a strong tree baseline, form residual seeds only from out-of-fold training predictions, and solve a graph-regularized quadratic with synchronized multi-right-hand-side preconditioned conjugate gradients. The graph weight and an analytic nonnegative blend are chosen on PBMC3k development cells and frozen before a PBMC4k transfer test. On PBMC3k, test MSE changes from 0.416046 to 0.398645; on PBMC4k it changes from 0.418608 to 0.412775. All five additional PBMC3k split audits favor the correction, but the cells are overlapping draws from one source and are not biological replicates. We prove existence, energy and perturbation bounds, the exact blend rule, and columnwise equivalence of synchronized and scalar PCG. Numerical checks support implementation correctness but are not proofs. A bounded twelve-right-hand-side audit measures a 1.21-fold scalar-to-synchronized solve ratio, while smaller blocks can be slower. The paper claims neither priority for residual propagation, donor generalization, biological recovery, nor an end-to-end speedup.

1. Audit outcome and scope. Single-cell matrices are sparse, but a graph-based estimator can still hide a dense cell–cell operator or a target leak. Both problems matter when the task is to predict a panel of measured genes from disjoint measured genes. The input transformation must not contain the withheld target values, and the linear algebra must expose what is stored and applied.

The initial manuscript failed a stronger-baseline audit. Its fixed 15-neighbor baseline had test MSE 0.4302 after a graph correction, whereas development-tuned ExtraTrees reached approximately 0.419 on the same split. That superiority claim is withdrawn. A second failure was subtler: dividing every cell by its total molecule count made each input depend on the twelve withheld target counts. This violates the paper's own information-flow statement even though the target columns were later removed.

The repaired study asks a narrower question. Can a graph-smooth correction of out-of-fold residuals improve a strong tabular baseline when (i) target counts are absent from input normalization, (ii) selection is separated from test targets in code, and (iii) the graph normal equation is applied without a dense $n \times n$ array? We test that question on two unmodified 10x Genomics PBMC archives released under CC BY 4.0 [1, 2, 23]. PBMC3k supplies development selection; all selected settings then transfer to PBMC4k.

This is a retrospective computational audit, not a preregistered experiment. PBMC3k and PBMC4k each contain one cross-sectional donor; neither supplies lineage, time, perturbation, or latent molecule truth. Consequently "recovery" means prediction of observed normalized entries, not biological state recovery, imputation of unobserved molecules, or causal inference. These distinctions are important because imputation can distort relationships even when a point metric improves [3, 20].

2. Literature boundary. Graph regularization and label propagation are established. Gaussian-field harmonic prediction and manifold regularization already combine labeled and unlabeled vertices through graph energies [4, 24]. Laplacian regression has statistical analyses under geometric assumptions that are not asserted here [8]. More directly, Jia and Benson model correlated regression residuals on graphs [14], and Correct and Smooth propagates base-model errors before smoothing predictions [13]. These papers collide with any claim that residual propagation itself is new.

The contribution here is therefore claim-separated rather than priority based: a target-excluded exposure for the masked-gene task; out-of-fold residual seeds for a flexible base learner; a closed-form development blend; an action-only normal equation; and a synchronized PCG implementation whose equivalence and cost are stated exactly. Cross-fitting, ExtraTrees, graph regularization, and multi-right-hand-side Krylov methods also have substantial prior histories [7, 16]. We do not call their combination the first of its kind.

Relevant expression-recovery methods include MAGIC, SAVER, and scVI [12, 15, 21]. They solve different tasks and require different likelihoods or training protocols, so they are not presented as

Table 1: Archive quantities recomputed from the two unchanged source files. The complete SHA-256 digests and URLs are stored in the JSON evidence.

Source	Genes	Cells	Nonzeros	Molecules	Bytes	SHA-256 prefix
10x Genomics PBMC3k	32,738	2,700	2,286,884	6,390,631	7,621,991	`847d6ebd9a1e`
10x Genomics PBMC4k	33,694	4,340	5,727,695	18,874,824	18,423,814	`097ff873bae6`

numerical baselines for this twelve-output supervised estimand. Instead we compare mean prediction, ridge, partial least squares (PLS), tuned kNN, random forests, and ExtraTrees [6, 7, 11, 19]. The tree baseline is essential: omitting it would recreate the rejected conclusion.

3. Authenticated data and leakage-safe protocol. Table 1 reports archive-level quantities, not copied website labels. The complete SHA-256 digests are

$$h_{3\mathrm{k}} = \texttt{847d6ebd9a1ec9a768f2be7e40ca42cb}\\ \texttt{fe75ebeb6d76a4c24167041699dc28b5},$$
$$h_{4\mathrm{k}} = \texttt{097ff873bae645bec5586898ac002cce}\\ \texttt{bbce952d52ac19ea0a35c27c4820df26}.$$

The loader rejects a digest, shape, nonzero count, or molecule count mismatch.

Let $C \in \mathbb{R}_{+}^{G\times n}$ be a count matrix and let $\mathcal{T}$ contain the twelve target-gene indices. For cell i, define the observable non-target exposure

$$s_i = \sum_{g\notin\mathcal{T}} C_{gi}, \qquad X_{gi} = \log\left(1 + 10^4 C_{gi}/s_i\right). \tag{3.1}$$

The same denominator normalizes target outcomes, but no target count enters an input coordinate through s_i. All s_i are positive in both archives.

PBMC3k is permuted with PCG64 seed 20260729 into 1620 fit, 540 development, and 540 test cells. The 96 input Ensembl IDs are the highest-variance eligible genes on fit cells among IDs present in both archives; all twelve target IDs and mitochondrial names are excluded. Standardization and a 16-component full-SVD PCA are fitted on the current training partition. The selected model is refit on the 2160 fit-plus-development cells before the PBMC3k test. PBMC4k uses a separate fixed-seed 3472/868 train/test split, the same Ensembl IDs, and settings frozen on PBMC3k. One feature changed annotation from TMEM66 to SARAF between references; its Ensembl ID remains fixed.

All tree models use one fitting thread. Reproduction commands also set the OpenMP, OpenBLAS, MKL, and NumExpr thread counts to one; this is part of the numerical protocol, not a timing optimization. Locked verification targets CPython 3.13.12 on macOS 26.6 arm64 with the version-locked PyPI numerical wheels and NumPy's Accelerate BLAS/LAPACK backend. Other binary backends can give different tree thresholds even when library version strings match. The public lock has its own recorded SHA-256 but is version-locked, not hash-locked.

The cell graph is transductive: it uses input covariates of test cells but never their targets. This is appropriate for recovery within a measured batch, not for an unseen-patient deployment claim. Five additional 2160/540 PBMC3k splits use frozen features and hyperparameters; they are descriptive sensitivity checks because their cells overlap.

4. Cross-fitted matrix-free residual propagation. Let $f(X)$ denote the selected ExtraTrees predictor and let $\widehat{y}_0$ be its full-training prediction. Split training indices into five folds. If cell i belongs to fold $a(i)$, its residual seed is

$$r_i = y_i - f^{(-a(i))}(X_i), \tag{4.1}$$

where $f^{(-a)}$ is trained without fold a. Development and test residual seeds are zero. This prevents a flexible learner's in-sample fit from artificially collapsing the correction signal.

On the PCA coordinates, store fifteen radial neighbors per cell. With median positive neighbor distance σ,

$$w_{ij} = \exp\left(-\frac{\|z_i - z_j\|_2^2}{2\sigma^2}\right), \qquad P_{ij} = \frac{w_{ij}}{\sum_{\ell\in\mathcal{N}(i)} w_{i\ell}}. \tag{4.2}$$

Thus P is row stochastic and has exactly $15n$ stored entries. Put $D = I - P$, let M be the diagonal training mask, and solve for all target columns U:

$$A_\lambda U = MR, \qquad A_\lambda = M + \mu I + \lambda D^\mathsf{T} D, \qquad \mu = 0.03. \tag{4.3}$$

The prediction is $\widehat{Y} = \widehat{Y}_0 + \alpha U$.

The graph-weight grid is $\lambda \in \{0.03, 0.1, 0.3, 1, 3, 10\}$. For each λ, development selection uses the exact nonnegative blend in Theorem 3; the best development MSE selects λ. PBMC3k chooses $(\lambda, \alpha) = (10, 1.000)$, which is then fixed for both final tests.

The implementation applies

$$A_\lambda V = (m + \mu \mathbf{1}) \odot V + \lambda\{(V - PV) - P^\mathsf{T}(V - PV)\} \tag{4.4}$$

to a dense block of twelve columns. It stores P in CSR form and never forms A_λ during recovery. Its Jacobi diagonal is

$$\operatorname{diag}(A_\lambda) = m + \mu \mathbf{1} + \lambda \left[\mathbf{1} + \operatorname{colsum}(P \odot P)\right]. \tag{4.5}$$

5. Mathematical guarantees. The statements below concern the declared finite-dimensional algorithm. They do not imply that graph smoothness is biologically correct.

Theorem 1 (Existence, uniqueness, and variational form)**.** *For $\mu > 0$ and $\lambda \geq 0$, A_λ is symmetric positive definite. For every residual matrix R, Eq.* (4.3) *has a unique solution, which uniquely minimizes*

$$J(V) = \tfrac{1}{2} \left\| M^{1/2}(V - R) \right\|_F^2 + \tfrac{\mu}{2} \|V\|_F^2 + \tfrac{\lambda}{2} \|DV\|_F^2 . \tag{5.1}$$

Proof. For every nonzero vector v,

$$v^\mathsf{T} A_\lambda v = \left\| M^{1/2} v \right\|_2^2 + \mu \|v\|_2^2 + \lambda \|Dv\|_2^2 \geq \mu \|v\|_2^2 > 0. \tag{5.2}$$

Thus A_λ is SPD. Differentiating Eq. (5.1) gives $\nabla J(V) = A_\lambda V - MR$. Its Hessian is the SPD operator A_λ acting on each column, so the stationary point is the unique minimizer and satisfies Eq. (4.3). □

Theorem 2 (Energy and seed-stability bounds)**.** *Let $U = A_\lambda^{-1} MR$. Then*

$$\mu \|U\|_F^2 + \lambda \|DU\|_F^2 \leq \|MR\|_F^2 . \tag{5.3}$$

If $\widetilde{U} = A_\lambda^{-1} M\widetilde{R}$, then

$$\left\| U - \widetilde{U} \right\|_F \leq \mu^{-1} \left\| M(R - \widetilde{R}) \right\|_F . \tag{5.4}$$

Proof. Optimality and feasibility of zero imply $J(U) \leq J(0) = \|MR\|_F^2 / 2$. Dropping the nonnegative fidelity term and multiplying by two proves Eq. (5.3). For the second claim, $A_\lambda(U - \widetilde{U}) = M(R - \widetilde{R})$. Since $\lambda_{\min}(A_\lambda) \geq \mu$,

$$\begin{aligned} \left\| U - \widetilde{U} \right\|_F &\leq \left\| A_\lambda^{-1} \right\|_2 \left\| M(R - \widetilde{R}) \right\|_F \\ &\leq \mu^{-1} \left\| M(R - \widetilde{R}) \right\|_F . \end{aligned} \tag{5.5}$$

□

Theorem 3 (Exact development blend)**.** *For fixed base error $E = Y_{\text{dev}} - \widehat{Y}_{0,\text{dev}}$ and correction $C = U_{\text{dev}}$, the minimizer of $\|E - \alpha C\|_F^2$ over $0 \leq \alpha \leq 1$ is*

$$\alpha^* = \Pi_{[0,1]} \left(\frac{\langle E, C \rangle_F}{\|C\|_F^2} \right), \tag{5.6}$$

with $\alpha^* = 0$ if $C = 0$. Its development loss cannot exceed the base loss.

Proof. Expanding gives $\|E\|_F^2 - 2\alpha\langle E, C\rangle_F + \alpha^2 \|C\|_F^2$. For $C \neq 0$ this strictly convex quadratic has unconstrained minimizer equal to the ratio in Eq. (5.6); Euclidean projection gives the constrained minimizer. If $C = 0$, every blend is equal and we choose zero. Since zero is feasible, the selected development loss is no larger than $\|E\|_F^2$. This is a development-set statement only; it gives no test-set guarantee. □

Theorem 4 (Graph perturbation)**.** *Let $L_P = (I - P)^\mathsf{T}(I - P)$ and $L_Q = (I - Q)^\mathsf{T}(I - Q)$, and let $U_P = (H + \lambda L_P)^{-1}B$ and $U_Q = (H + \lambda L_Q)^{-1}B$, where $H = M + \mu I$. Then*

$$\|U_P - U_Q\|_F \leq \frac{\lambda}{\mu^2} \|L_P - L_Q\|_2 \|B\|_F . \tag{5.7}$$

Proof. Write $A_P = H + \lambda L_P$. Subtracting the two systems gives $A_P(U_P - U_Q) = \lambda(L_Q - L_P)U_Q$. Both normal matrices have smallest eigenvalue at least μ, so

$$\|U_P - U_Q\|_F \leq \frac{\lambda}{\mu} \|L_Q - L_P\|_2 \|U_Q\|_F , \tag{5.8}$$

$$\|U_Q\|_F \leq \mu^{-1} \|B\|_F . \tag{5.9}$$

Combining the inequalities proves Eq. (5.7). □

Theorem 5 (Synchronized PCG equivalence and cost)**.** *In exact arithmetic, synchronized PCG applied to q columns with separate columnwise inner products produces, for every active column, the same iterates as scalar Jacobi-PCG with the same start. If scalar column g stops after m_g iterations and $m_{\max} = \max_g m_g$, synchronized execution uses $m_{\max}$ block operator applications, versus $\sum_g m_g$ scalar applications. For $\mathrm{nnz}(P) \leq nk$, its storage is $O(nk + nq)$ and its arithmetic is $O(nkqm_{\max})$ in the dense-block implementation.*

Proof. At any iteration, Eq. (4.4) applied to a block has column g equal to $A_\lambda p_g$. The code computes the PCG scalars

$$a_g = \frac{r_g^\mathsf{T} z_g}{p_g^\mathsf{T} A_\lambda p_g}, \qquad b_g = \frac{(r_g^+)^\mathsf{T} z_g^+}{r_g^\mathsf{T} z_g} \tag{5.10}$$

independently for each column. Hence the updates $u_g^+ = u_g + a_g p_g$, $r_g^+ = r_g - a_g A_\lambda p_g$, and $p_g^+ = z_g^+ + b_g p_g$ are exactly the scalar recurrences. Induction from the common zero start proves equivalence until each column stops. One synchronized operator call advances all active columns, giving the launch counts. Each call uses P and P^T on a dense $n \times q$ block, so its arithmetic is $O(nkq)$; storing CSR plus the block vectors costs $O(nk + nq)$. □

Corollary 1 (Residual certificate)**.** *If $\widetilde{U}$ has residual $F = MR - A_\lambda \widetilde{U}$, then*

$$\left\|U - \widetilde{U}\right\|_F \leq \mu^{-1} \|F\|_F . \tag{5.11}$$

Proof. $U - \widetilde{U} = A_\lambda^{-1} F$ and $\left\|A_\lambda^{-1}\right\|_2 \leq 1/\mu$ by Theorem 1. □

Theorem 5 reduces sparse-operator launches, not asymptotic arithmetic. A matrix–block kernel can be slower for small q and faster for larger q; that crossover must be measured rather than inferred.

6. Results. Figure 1 and Table 2 show the required strong-baseline correction. ExtraTrees is the best uncorrected method on both tests. CF-MFRP lowers PBMC3k MSE by 4.18% and PBMC4k MSE by 1.39%. These are two dataset-specific observations. They do not establish universal superiority over tree ensembles or specialist single-cell methods.

The correction wins for 9 of 12 targets on PBMC3k and 11 of 12 on PBMC4k (Fig. 2, Table 3). The losing genes are retained. No marker is converted into a cell-type or functional conclusion. Figure 3 isolates development-only selection, and Fig. 4 retains the sign and magnitude of all five split audits.

All five repeated splits favor the correction (5 of 5), with relative MSE changes ranging from approximately -0.7% to -3.8%. This reduces concern that the sign is unique to one cell permutation, but it does not add donors.

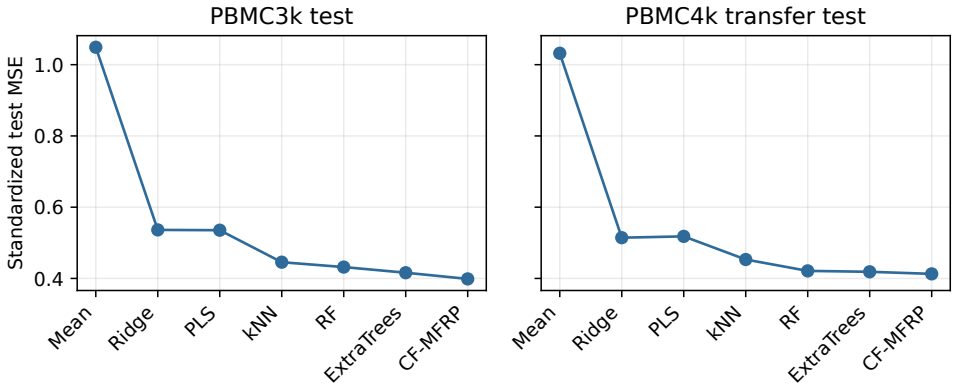


Fig. 1: Leakage-safe test MSE for every selected baseline and CF-MFRP. All hyperparameters were selected on PBMC3k development cells; PBMC4k uses their frozen values. The former graph method is absent because the audit rejected it.

Table 2: Test metrics after family-wise development selection on PBMC3k. Lower is better. PBMC4k is a frozen-parameter transfer, not a donor-population test.

Method	PBMC3k MSE	PBMC3k MAE	PBMC4k MSE	PBMC4k MAE
Mean	1.0489	0.7097	1.0321	0.6983
Ridge	0.5362	0.4584	0.5143	0.4558
PLS	0.5353	0.4618	0.5180	0.4591
kNN	0.4455	0.3410	0.4531	0.3628
Random forest	0.4319	0.3648	0.4212	0.3728
ExtraTrees	0.4160	0.3620	0.4186	0.3717
CF-MFRP	0.3986	0.3350	0.4128	0.3521

The PBMC3k graph stores 1.148% of the value-plus-index scalar count of a dense matrix; PBMC4k stores 0.714%. Maximum relative residuals are 9.63e-10 and 9.60e-10. Explicit dense controls at $n \in \{64, 128, 256\}$ agree with the matrix-free solution to at worst 1.90×10^{-10}. This validates implementation of the declared equation, not the graph model. Table 4 gives the storage and residual certificates supporting that implementation check.

Figure 5 is deliberately not summarized as a universal speedup. At twelve right-hand sides the measured scalar-to-synchronized ratio is 1.21, but at one, two, and four right-hand sides the synchronized Python/SciPy path is slower. Threading, cache, sparse layout, and hardware can move the crossover. The robust computational claims are the storage bound, operator-launch count, residual, and scalar equivalence.

7. Reproducibility and limitations. The public package contains both original CC BY 4.0 archives, their SHA-256 digests, source attribution, deterministic seeds, selected Ensembl IDs, all tuning rows without test metrics, fold-level zero-overlap audits, locked JSON, generated TeX tables, figure code, and tests. The principal commands are

```
python -m pip install --only-binary=:all: \
  -r code/requirements-lock.txt
python -m pip install --no-deps ./code
mfcell-reproduce --verify
python code/scripts/validate_release.py
```

Clean replay within the registered reference environment compares discrete protocol fields and digests exactly and floating evidence within relative tolerance 0.5% and absolute tolerance 5×10^{-5}; environment and runtime fields are excluded. It recomputes archive statistics, feature selection, model selection, both final tests, repeated splits, dense controls, gene-level rank correlations, and scalar–batched agreement.

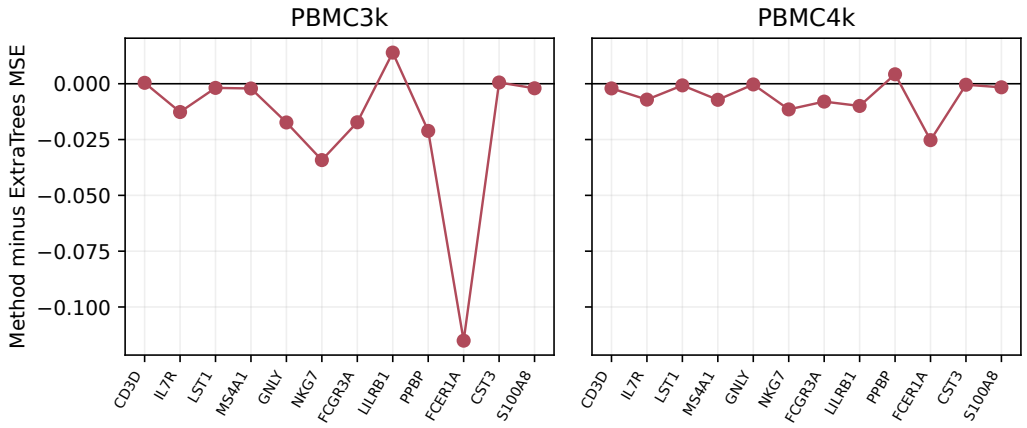


Fig. 2: Gene-resolved change relative to ExtraTrees. Values below zero favor the correction. Aggregate improvement coexists with target-specific losses.

Table 3: Gene-level MSE. ET denotes ExtraTrees; CF denotes CF-MFRP; Δ is corrected minus ExtraTrees MSE.

Gene	3k ET	3k CF	Δ3k	4k ET	4k CF	Δ4k
CD3D	0.4478	0.4482	+0.0004	0.3544	0.3523	-0.0021
IL7R	0.6558	0.6431	-0.0127	0.5576	0.5505	-0.0071
LST1	0.1929	0.1910	-0.0019	0.1911	0.1904	-0.0007
MS4A1	0.3398	0.3377	-0.0021	0.2223	0.2151	-0.0072
GNLY	0.4941	0.4768	-0.0173	0.5137	0.5134	-0.0003
NKG7	0.3336	0.2994	-0.0342	0.2431	0.2316	-0.0115
FCGR3A	0.3296	0.3124	-0.0173	0.3285	0.3205	-0.0080
LILRB1	0.7319	0.7459	+0.0139	0.8241	0.8142	-0.0100
PPBP	0.4056	0.3845	-0.0211	1.1220	1.1262	+0.0042
FCER1A	0.7376	0.6226	-0.1151	0.3124	0.2871	-0.0253
CST3	0.1750	0.1756	+0.0006	0.1825	0.1821	-0.0005
S100A8	0.1487	0.1467	-0.0020	0.1714	0.1698	-0.0016

The limitations are material. First, the test outcomes were observed during this repair, so this is not prospective confirmation. Second, two donors are not a donor population; PBMC4k and PBMC3k also differ in reference build and Cell Ranger version. Third, a random cell split is easier than a donor-held-out clinical task. Fourth, target-excluded library normalization defines a clear masked-gene estimand but is not asserted to be optimal single-cell normalization. Fifth, ExtraTrees and the listed simple baselines are strong for this sample size, yet the experiment does not exhaust all probabilistic or deep single-cell models. Sixth, standardized MSE can reward shrinkage and does not measure covariance preservation or latent molecule truth. Seventh, the graph uses test covariates and is transductive. Finally, synchronized PCG changes launch granularity, not the mathematical estimator or worst-case arithmetic order.

The next defensible study would freeze this protocol before acquiring multiple donors, evaluate donor-held-out panels and covariance distortion, and compare against specialist count models under matched tuning budgets. Until then, the evidence supports a reproducible two-source numerical finding and the stated finite-dimensional bounds—nothing broader.

A. Target-label noninterference.

Proposition 1 (Sealed test targets)**.** *After PBMC3k development selection is frozen, each test prediction is a deterministic function of public constants, all non-target covariates, training targets, fold assignments, and the graph; it is independent of test-target entries as program inputs.*

Proof. Equation (3.1) removes every target count from the exposure, so input normalization is a func-

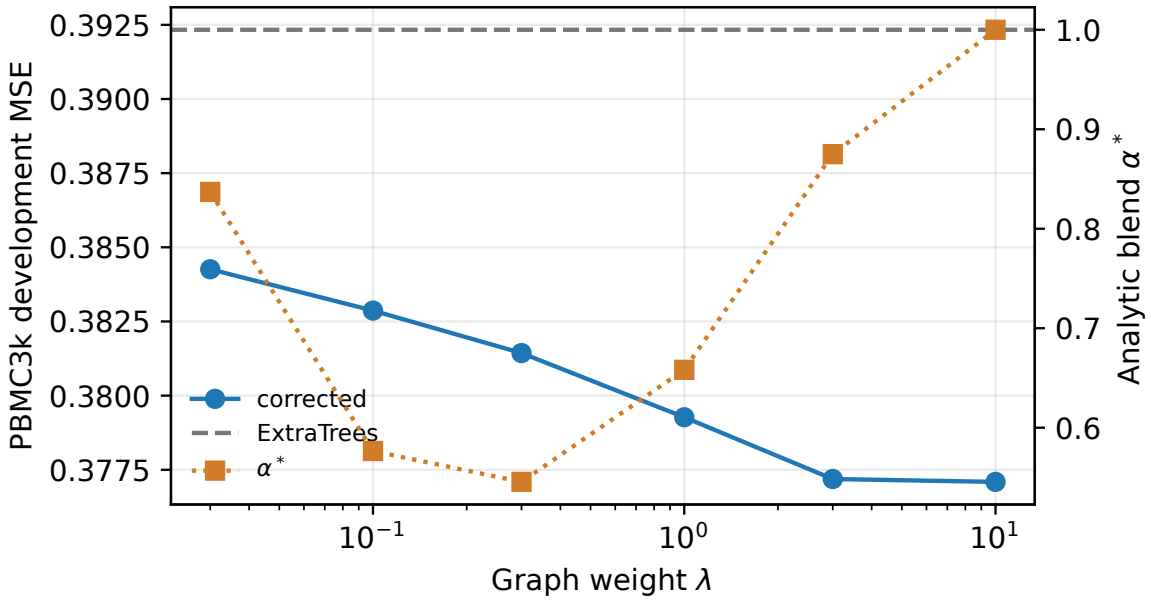


Fig. 3: PBMC3k development-only selection. The blend is computed analytically for each graph weight; test errors are not present in this artifact.

Table 4: Sparse-operator and PCG certificates for the two final recovery runs.

Source	n	nnz(P)	CSR/dense	Iterations	Max residual
pbmc3k	2700	40500	0.0115	49	9.63e-10
pbmc4k	4340	65100	0.0071	50	9.60e-10

tion of non-target counts alone. Input-gene selection, standardization, PCA, and graph construction consume only those inputs and training indices. The full base learner consumes training targets; each out-of-fold seed additionally excludes its own fold targets from its fitted model. Equation (4.3) has right side MR, which is zero outside training cells. The frozen λ and α are constants on the final test runs. Composition of these maps therefore contains no test-target argument. Test targets enter only the subsequent metric computation. □

This is an information-flow proof for the implementation contract. It does not make test cells statistically independent, because their graph covariates and biological source are shared.

B. Conditioning and classical PCG bound. Because $0 \preceq M \preceq I$ and $\|D\|_2 \le 1 + \|P\|_2$,

$$\lambda_{\max}(A_\lambda) \le 1 + \mu + \lambda(1 + \|P\|_2)^2, \tag{B.1}$$

whereas $\lambda_{\min}(A_\lambda) \ge \mu$. Therefore

$$\kappa_2(A_\lambda) \le \frac{1 + \mu + \lambda(1 + \|P\|_2)^2}{\mu}. \tag{B.2}$$

For unpreconditioned CG, the standard energy-norm estimate is

$$\frac{\|e_m\|_{A_\lambda}}{\|e_0\|_{A_\lambda}} \le 2\left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}\right)^m. \tag{B.3}$$

The implementation uses a Jacobi preconditioner and reports actual residuals; it does not substitute this conservative bound for a measured stopping test [5, 10, 18].

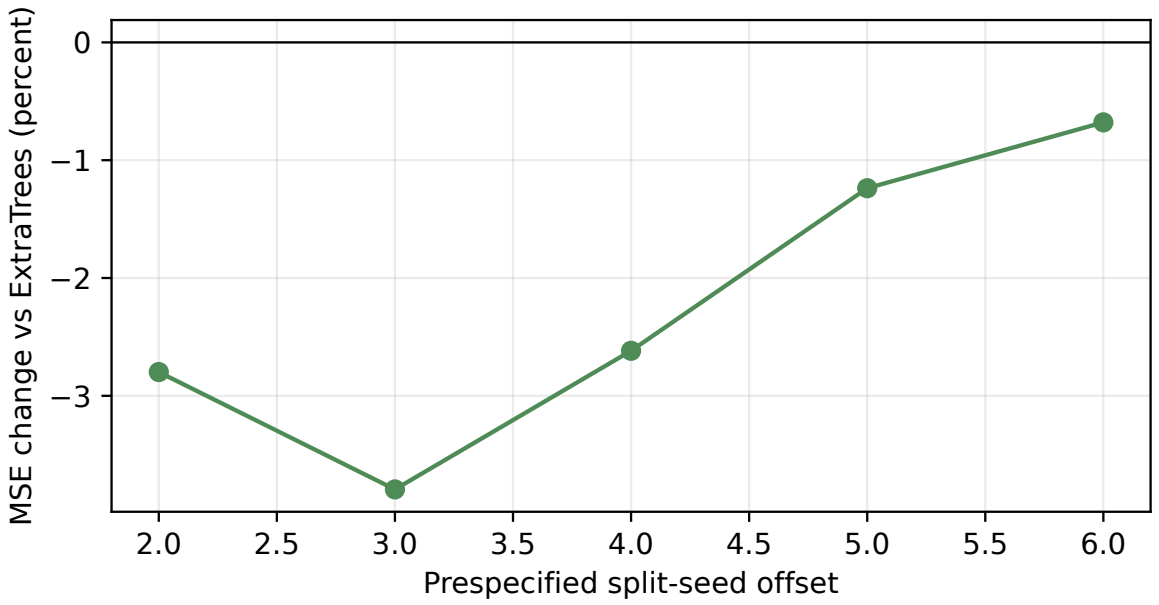


Fig. 4: Five frozen-parameter PBMC3k split audits. Negative values favor CF-MFRP. Overlapping cells make these sensitivity checks, not independent replicates or confidence intervals.

C. Software and environment boundary. NumPy, SciPy, scikit-learn, and Matplotlib provide the array, sparse solve, baseline, and plotting layers [9, 17, 22]. Random seeds fix splits, forests, and folds. Scientific floats are compared within declared tolerances across a fresh replay. Wall times are excluded because they depend on the current Python, library, CPU, thread, and power state. The release records those fields rather than hiding them.

References.

[1] 10x Genomics. 3k PBMCs from a healthy donor, 2016. URL https://www.10xgenomics.com/datasets/3-k-pbm-cs-from-a-healthy-donor-1-standard-1-1-0. Cell Ranger 1.1.0; CC BY 4.0; accessed 31 July 2026.

[2] 10x Genomics. 4k PBMCs from a healthy donor, 2017. URL https://www.10xgenomics.com/datasets/4-k-pbm-cs-from-a-healthy-donor-2-standard-2-1-0. Cell Ranger 2.1.0; CC BY 4.0; accessed 31 July 2026.

[3] Tallulah S. Andrews and Martin Hemberg. False signals induced by single-cell RNA-seq imputation. *F1000Research*, 7:1740, 2018. doi: 10.12688/f1000research.16613.2.

[4] Mikhail Belkin, Partha Niyogi, and Vikas Sindhwani. Manifold regularization: A geometric framework for learning from labeled and unlabeled examples. *Journal of Machine Learning Research*, 7:2399–2434, 2006. URL https://www.jmlr.org/papers/v7/belkin06a.html.

[5] Michele Benzi. Preconditioning techniques for large linear systems: A survey. *Journal of Computational Physics*, 182(2):418–477, 2002. doi: 10.1006/jcph.2002.7176.

[6] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.

[7] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63: 3–42, 2006. doi: 10.1007/s10994-006-6226-1.

[8] Alden Green, Sivaraman Balakrishnan, and Ryan J. Tibshirani. Minimax optimal regression over sobolev spaces via laplacian regularization on neighborhood graphs. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130, pages 2602–2610. PMLR, 2021. URL https://proceedings.mlr.press/v130/green21a.html.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.

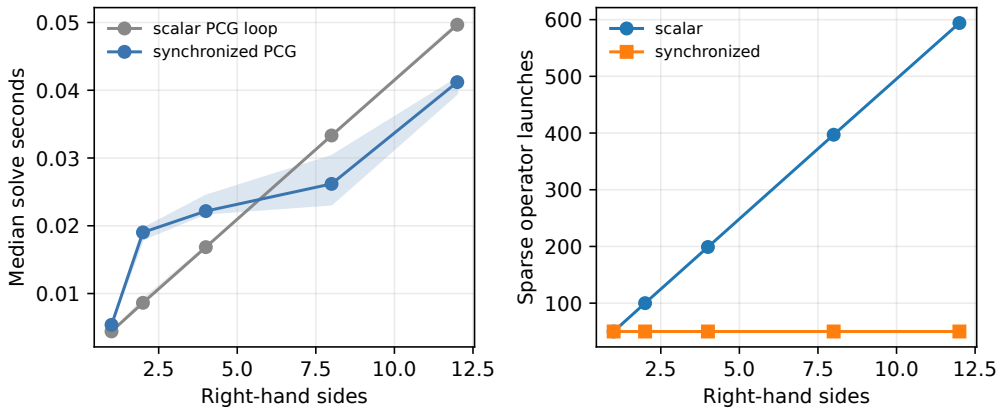


Fig. 5: Bounded PBMC3k solve audit: medians and interquartile ranges over seven alternating-order repetitions (left), and sparse operator launches (right). Synchronized PCG loses for one to four right-hand sides and wins for eight and twelve on this environment. The twelve-column solutions agree with scalar PCG to 2.66e-16.

[10] Magnus R. Hestenes and Eduard Stiefel. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49:409–436, 1952. doi: 10.6028/jres.049.044.

[11] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

[12] Mo Huang, Jingshu Wang, Eduardo Torre, Hannah Dueck, Sydney Shaffer, Roberto Bonasio, John I. Murray, Arjun Raj, Mingyao Li, and Nancy R. Zhang. SAVER: gene expression recovery for single-cell RNA sequencing. *Nature Methods*, 15:539–542, 2018. doi: 10.1038/s41592-018-0033-z.

[13] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin R. Benson. Combining label propagation and simple models out-performs graph neural networks. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.13993.

[14] Junteng Jia and Austin R. Benson. Residual correlation in graph neural network regression. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 588–598, 2020. doi: 10.1145/3394486.3403101.

[15] Romain Lopez, Jeffrey Regier, Michael B. Cole, Michael I. Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15:1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.

[16] Dianne P. O'Leary. The block conjugate gradient algorithm and related methods. *Linear Algebra and its Applications*, 29:293–322, 1980. doi: 10.1016/0024-3795(80)90247-5.

[17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[18] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2 edition, 2003. doi: 10.1137/1.9780898718003.

[19] Charles J. Stone. Consistent nonparametric regression. *The Annals of Statistics*, 5(4):595–620, 1977. doi: 10.1214/aos/1176343886.

[20] Valentine Svensson. Droplet scRNA-seq is not zero-inflated. *Nature Biotechnology*, 38:147–150, 2020. doi: 10.1038/s41587-019-0379-5.

[21] David van Dijk, Roshan Sharma, Juozas Nainys, Kristina Yim, Pooja Kathail, Ambrose J. Carr, Cassandra Burdziak, Kevin R. Moon, Christine L. Chaffer, Diwakar Pattabiraman, Brian Bierie, Linas Mazutis, Guy Wolf, Smita Krishnaswamy, and Dana Pe'er. Recovering gene interactions from single-cell data using data diffusion. *Cell*, 174(3):716–729.e27, 2018. doi: 10.1016/j.cell.2018.05.061.

[22] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, Ilhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, and Paul van Mulbregt. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[23] Grace X. Y. Zheng, Jessica M. Terry, Phillip Belgrader, Paul Ryvkin, Zachary W. Bent, Ryan Wilson, Solongo B. Ziraldo, Tobias D. Wheeler, Geoff P. McDermott, Junjie Zhu, Mark T. Gregory, Joe Shuga, Luz Montesclaros, Jason G. Underwood, Donald A. Masquelier, Stefanie Y. Nishimura, Michael Schnall-Levin, Paul W. Wyatt, Christopher M. Hindson, Rajiv Bharadwaj, Alexander Wong, Kevin D. Ness, Lan W. Beppu, H. Joachim Deeg, Christopher McFarland, Keith R. Loeb, William J. Valente, Nolan G. Ericson, Emily A. Stevens, Jerald P. Radich, Tarjei S. Mikkelsen, Benjamin J. Hindson, and Jason H. Bielas. Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8:14049, 2017. doi: 10.1038/ncomms14049.

[24] Xiaojin Zhu, Zoubin Ghahramani, and John D. Lafferty. Semi-supervised learning using gaussian fields and harmonic functions. In *Proceedings of the Twentieth International Conference on Machine Learning*, pages 912–919, 2003. URL https://www.cs.cmu.edu/~zhuxj/pub/zgl.pdf.